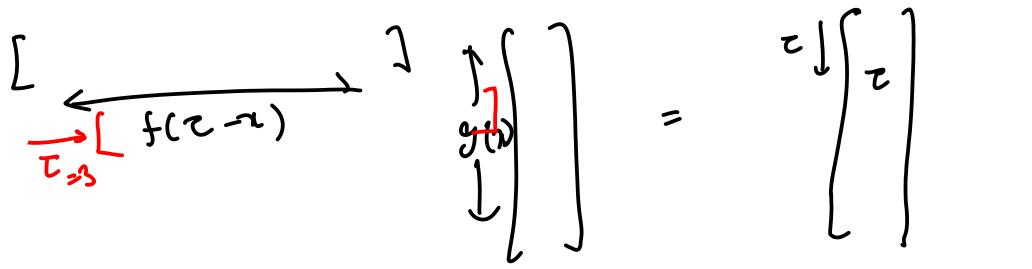
CNN: Convolutional Neural Networks

$$(f * g)(\tau) = \int_{x=-\infty}^{\infty} f(\tau - x)\, g(x)\, dx$$

$$= \sum_{x=mm}^{max} f[\tau - x] \cdot g[x]$$

$(f * g)(\tau)$



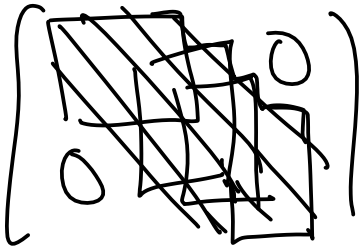

$$(f * g)(\tau) = \begin{bmatrix} f(\tau - x) \end{bmatrix}^{T} \begin{bmatrix} g(x) \end{bmatrix}$$

$f(0-x)$

$$(f * g)(\tau) = \begin{bmatrix} w & 0 & 0 & 0 & 0 \\ 0 & w & f(1-x) & \cdots & \\ 0 & 0 & w f(2-x) & \cdots & \\ & & & & \\ 0 & \cdots & f(n-x) & & \end{bmatrix} \begin{bmatrix} g(x) \\ \vdots \\ \end{bmatrix} = \begin{bmatrix} \\ \\ \end{bmatrix} \leftarrow (f * g)(\tau)$$

Weight matrix; ↑ input; 1D convolution

Discrete convolution can be written as matrix multiplication.
Convolution is a linear operation in each of the input functions.
Convolution Layer is a specific kind of Linear layer.



1. Convolution 2D operations on image can be interpretted as:
a. Edge detection
b. Patch matching

When we learn the convolution kernels, we expect them to learn edges/patches that repeat in the dataset and are informative for the task.

2. Convolution neural networks are well suited for images, and you have to combine them with Pooling layers
3. CNNs are pyramidic in structure with

with decreasing number of dimensions

1. Why are use using Conv layers in the first place? Adv. / dis. with respect to Linear layers?

Answer: Conv layers reduce the number of weights by a large amount as compared to linear layers at the (Cost of/benefit of) translation invariance